Solving Problems by Searching

Dr. Oybek Eraliev, Phd

Department of Computer Engineering

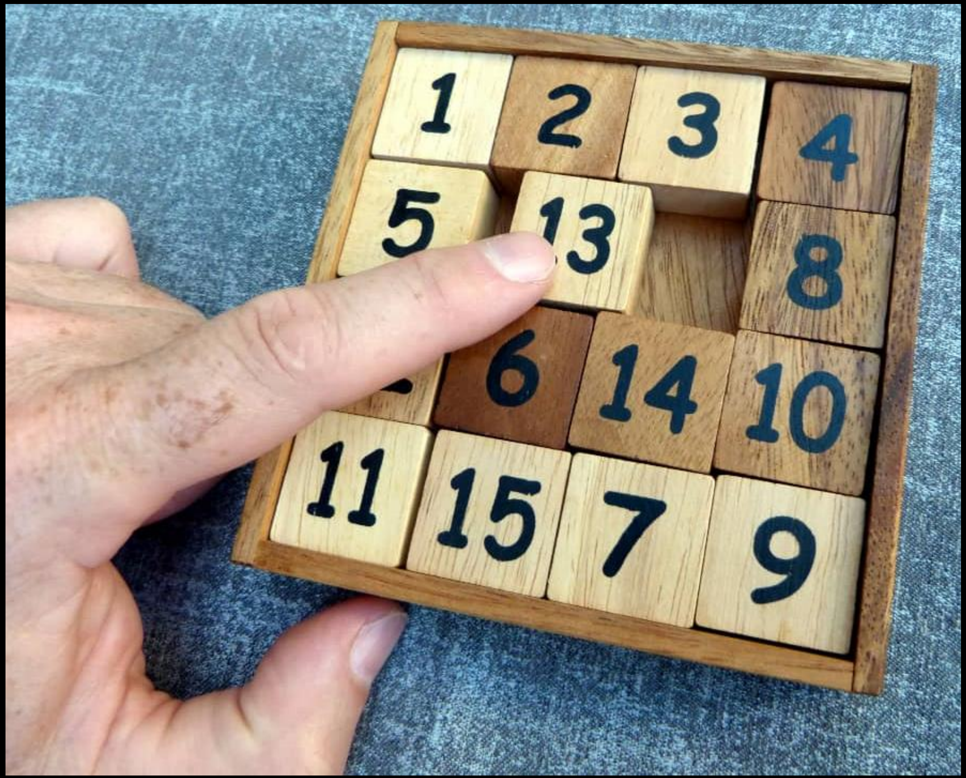
Inha University In Tashkent.

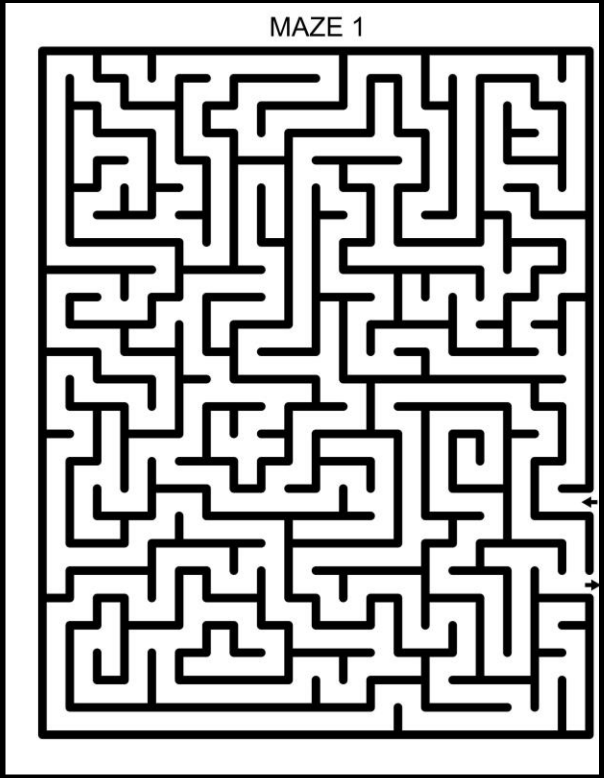
Email: oybekeraliev7@gmail.com

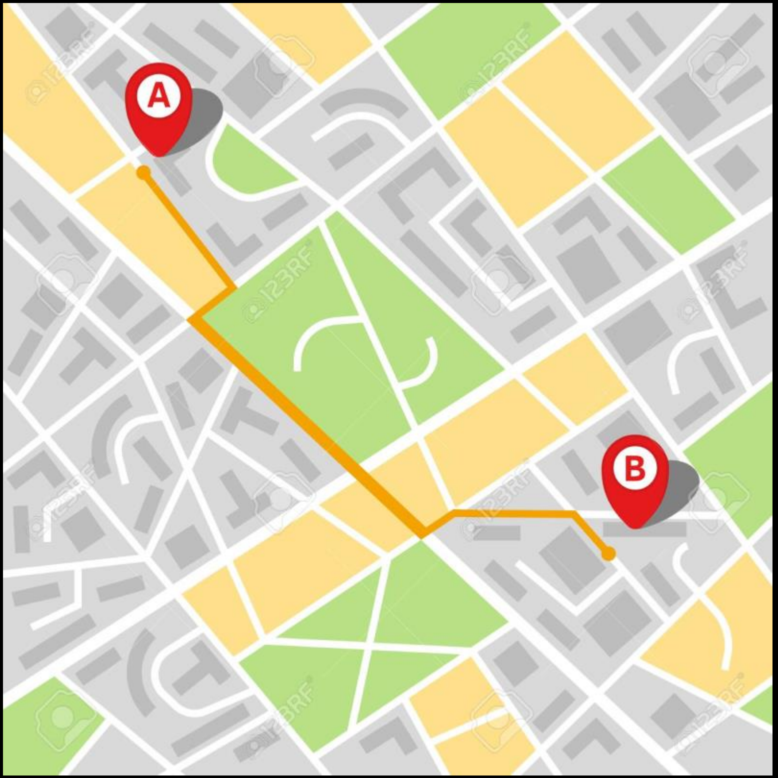
Content

- Search Algorithms (terminology)
- Uninformed Search Strategies
- Informed (Heuristic) Search Strategies

Search Algorithms (Applications)







Search Algorithms (Terminology)

agent

Entity that perceives its environment and acts upon that environment

Agent can be :

- person,
- algorithm (abstract)
- machine (phisical machine like robot)

Search Algorithms (Terminology)

state

A configuration of the agent and its environment

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	

2	1	6	7
5	3	4	9
8	12	11	10
15	14	13	

Search Algorithms (Terminology)

Initial state

2	1	6	7
5	3	4	9
8	12	11	10
15	14	13	

15	1	6	13
5	3	4	9
8	12	11	10
7	14	2	

5	1	6	13
15	12	4	11
8	3	9	10
7	14	2	

Search Algorithms (Terminology)

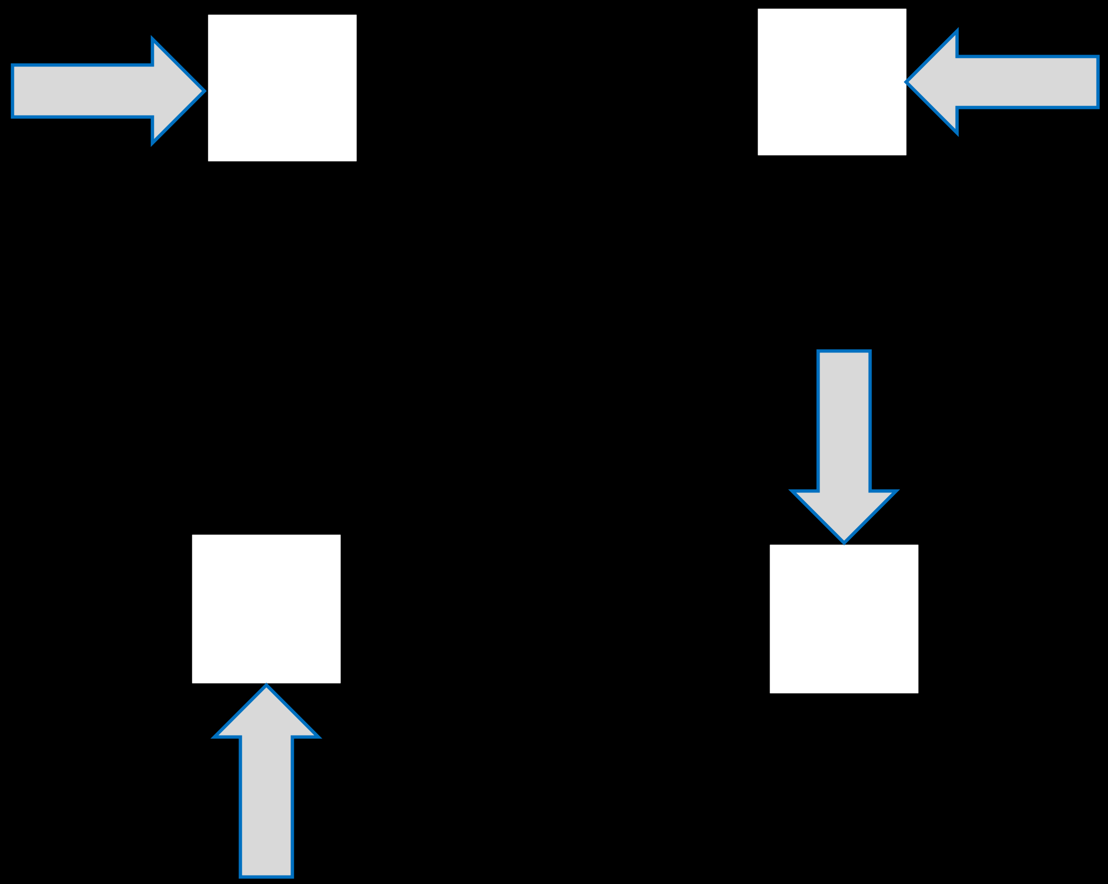
action

Choices that can be made in a state

$ACTIONS(s)$ returns the set of actions that can be executed in state s

Search Algorithms (Terminology)

actions



Search Algorithms (Terminology)

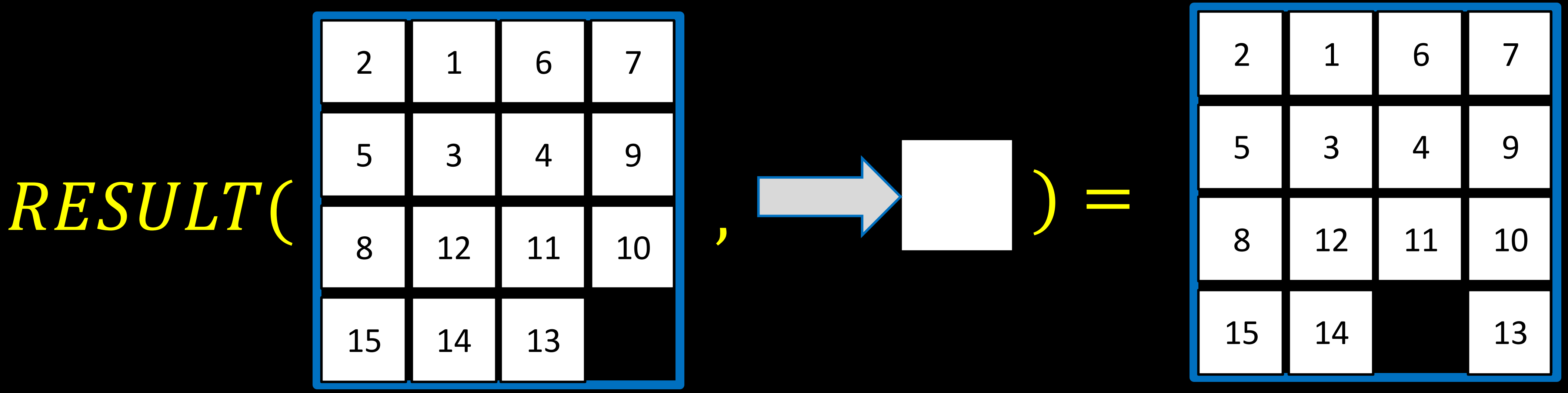
transition model

A description of what state results from performing any applicable action in any state

$RESULT(s, a)$ returns the state resulting from performing action a in state s

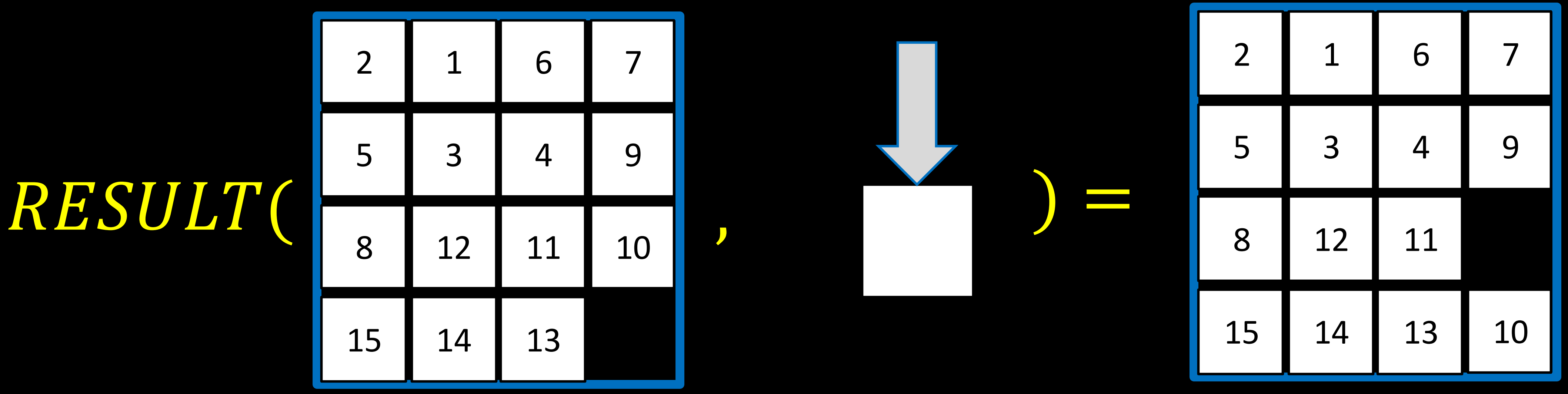
Search Algorithms (Terminology)

transition model



Search Algorithms (Terminology)

transition model



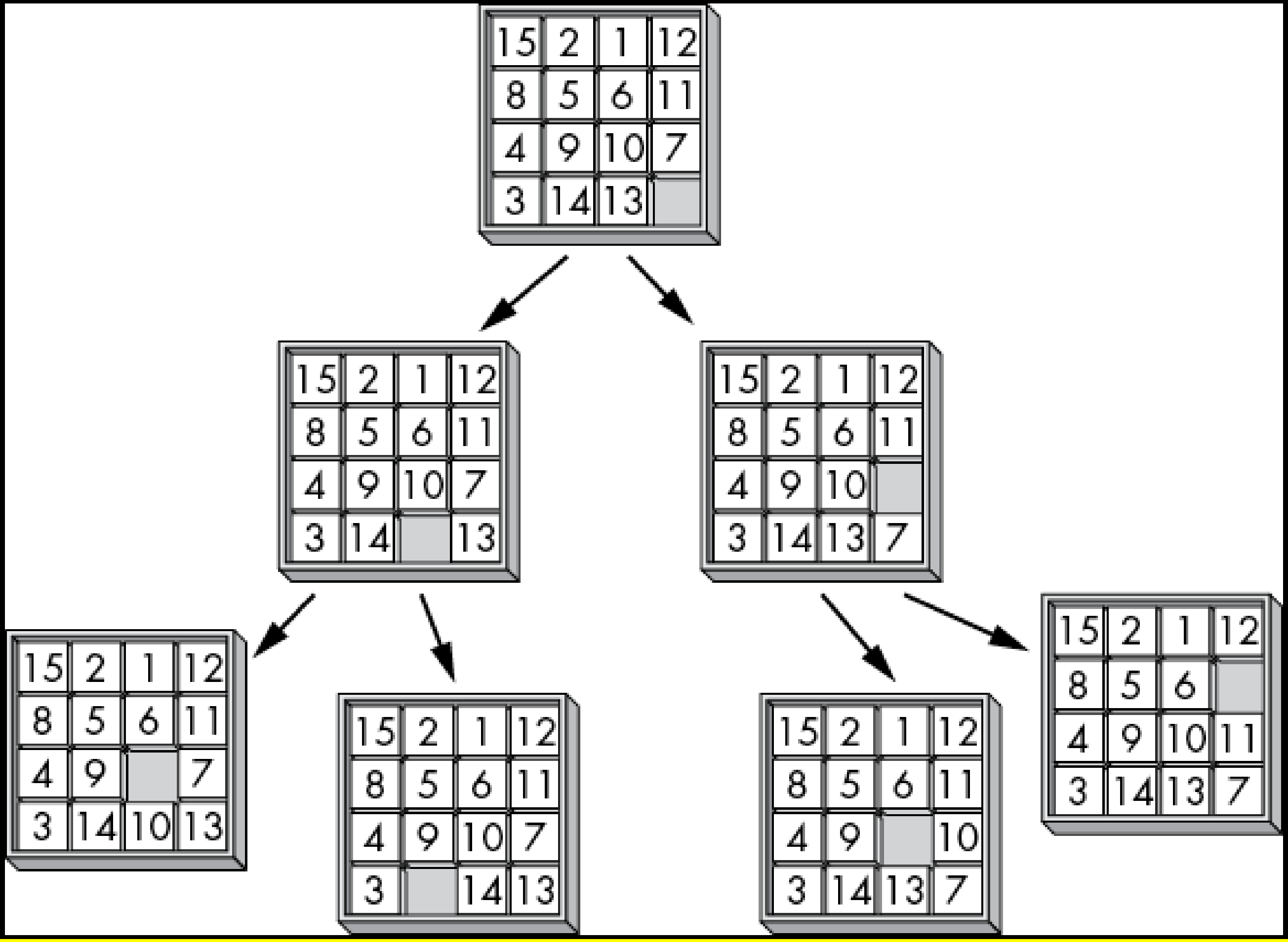
Search Algorithms (Terminology)

state space

the set of all states reachable from the initial state by any sequences of actions

Search Algorithms (Terminology)

state space



Search Algorithms (Terminology)

goal test

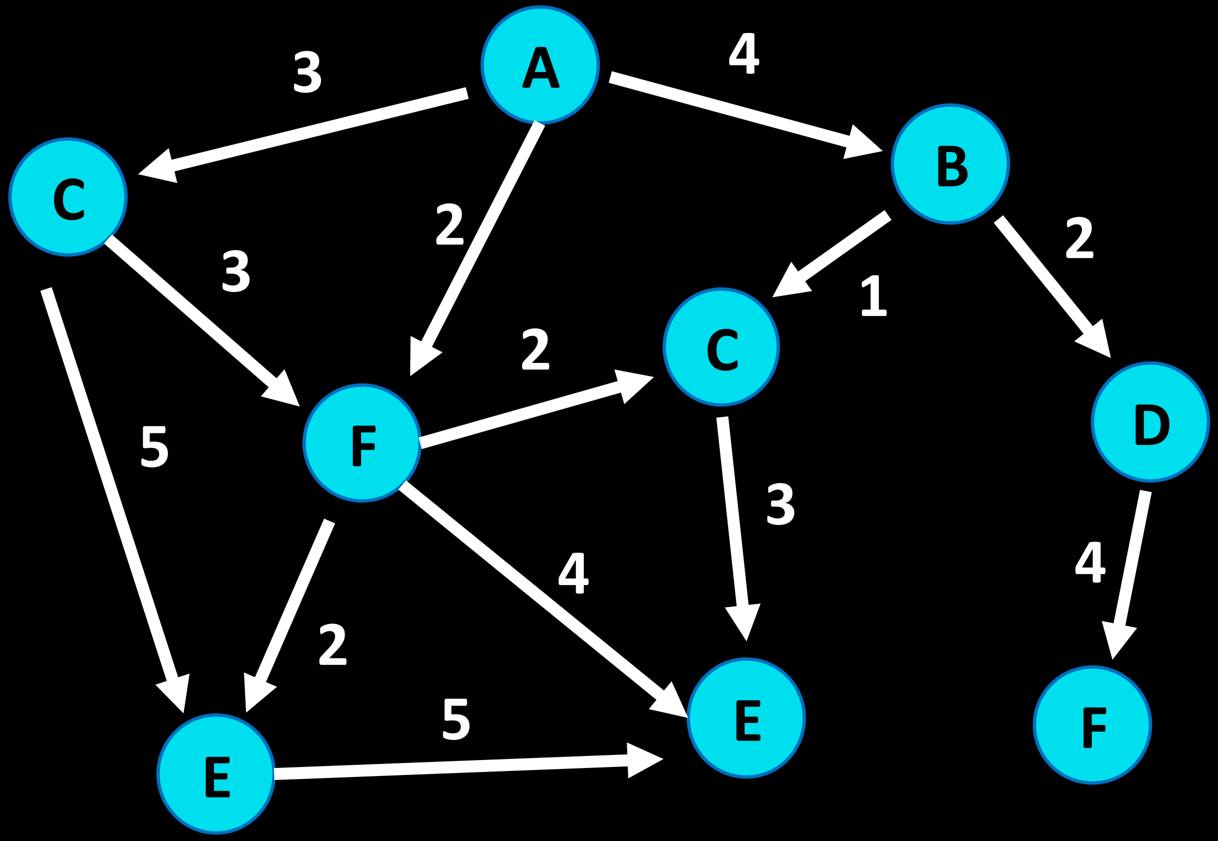
way to determine whether a given state is a goal state

Search Algorithms (Terminology)

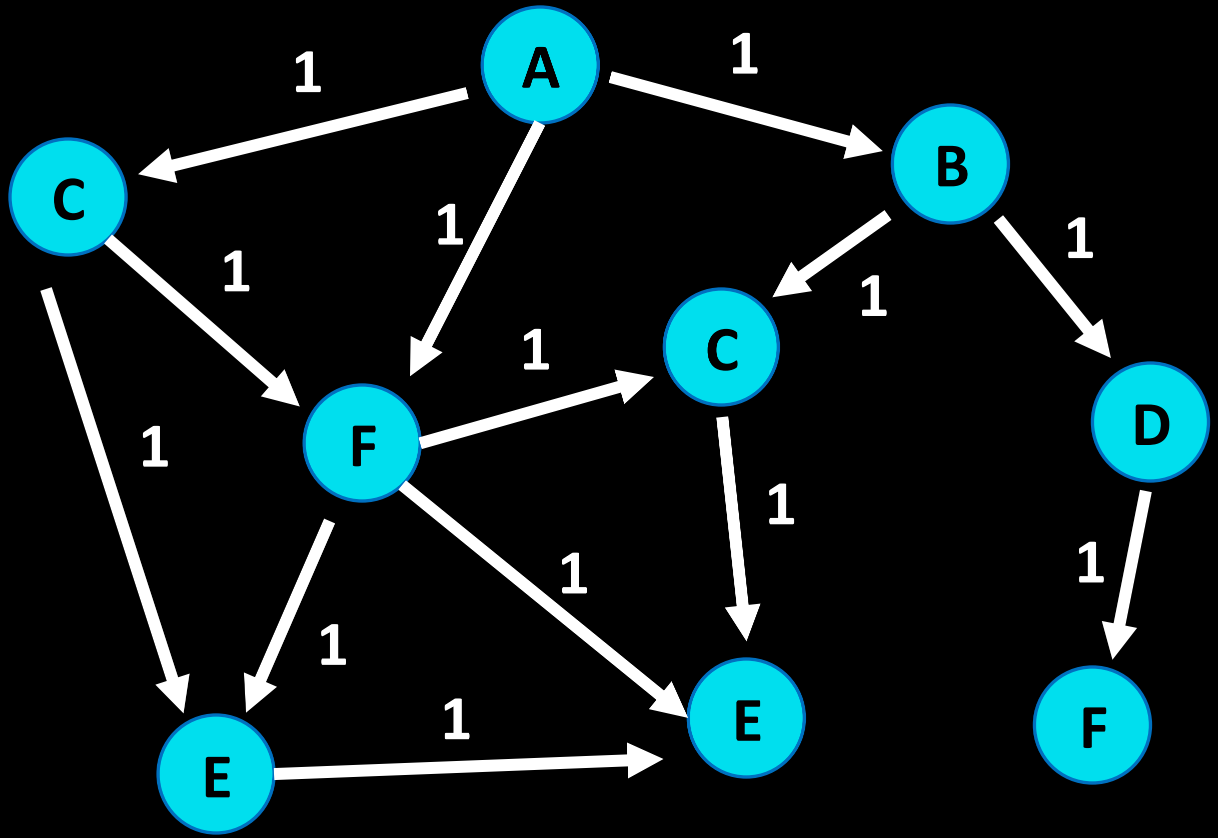
path cost

numerical cost associated with a given path

Search Algorithms (Terminology)



Search Algorithms (Terminology)



Search Algorithms (Terminology)

initial state

actions

transition model

goal test

path cost function

Search Algorithms (Terminology)

solution

A sequence of actions that leads from the initial state to a goal state

Search Algorithms (Terminology)

optimal solution

A solution that has the lowest path cost among all solutions

Search Algorithms (Terminology)

node

a data structure that keeps track of

- a state
- a parent (node that generate this node)
- an action (action applied to parent to get node)
- a path cost (from initial state to node)

Search Algorithms

- A search algorithm takes a search problem as input and returns a solution, or an indication of failure.
- We consider algorithms that superimpose a search tree over the state-space graph, forming various paths from the initial state, trying to find a path that reaches a goal state.
- Each node in the search tree corresponds to a state in the state space and the edges in the search tree correspond to actions. The root of the tree corresponds to the initial state of the problem.

Search Algorithms

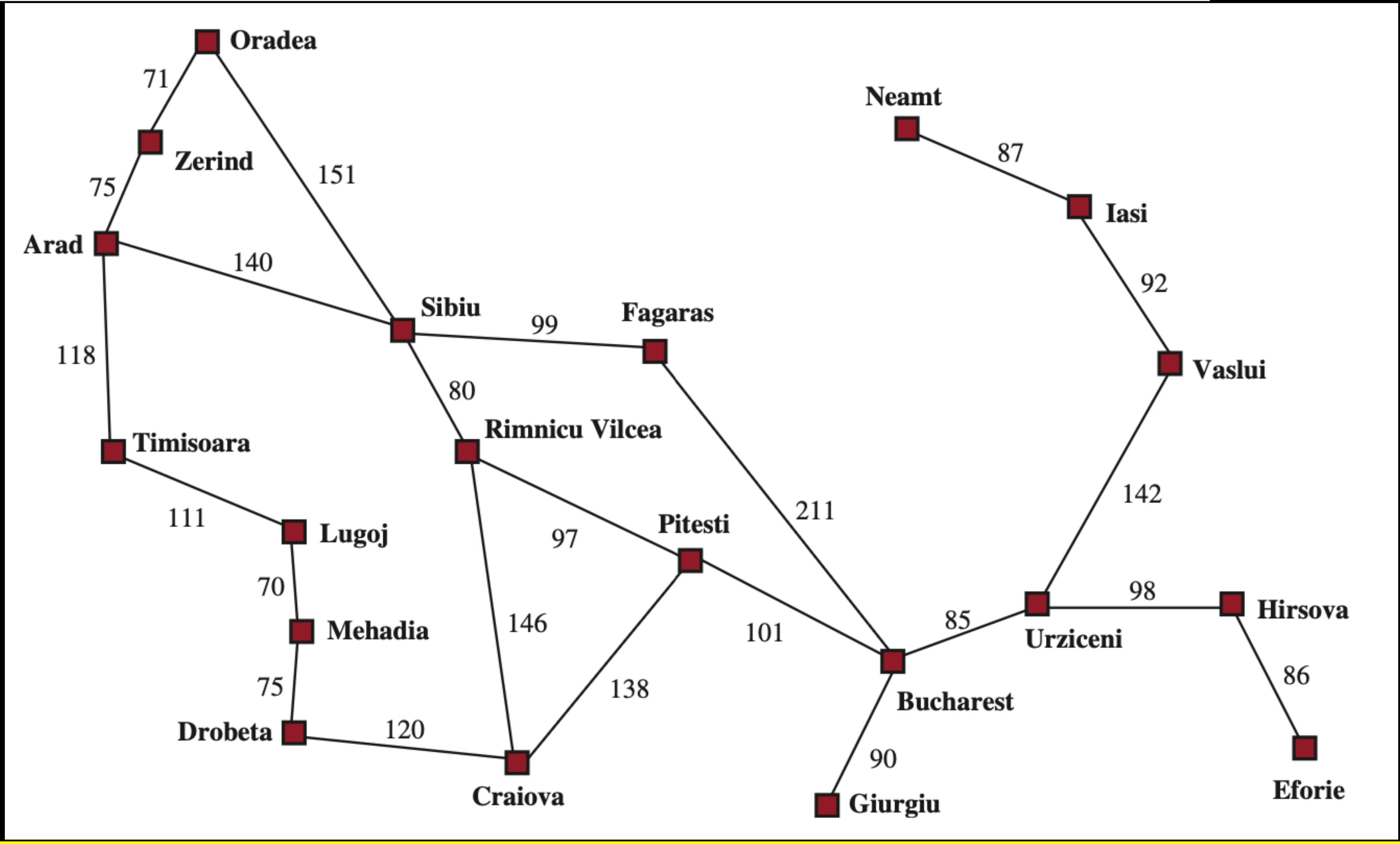
- It is important to understand the distinction between the **state space** and the **search tree**.

- The **state space** describes the set of states in the world, and the actions that allow transitions from one state to another.

- The **search tree** describes paths between these states, reaching towards the goal. The search tree may have multiple paths to any given state, but each node in the tree has a unique path back to the root.

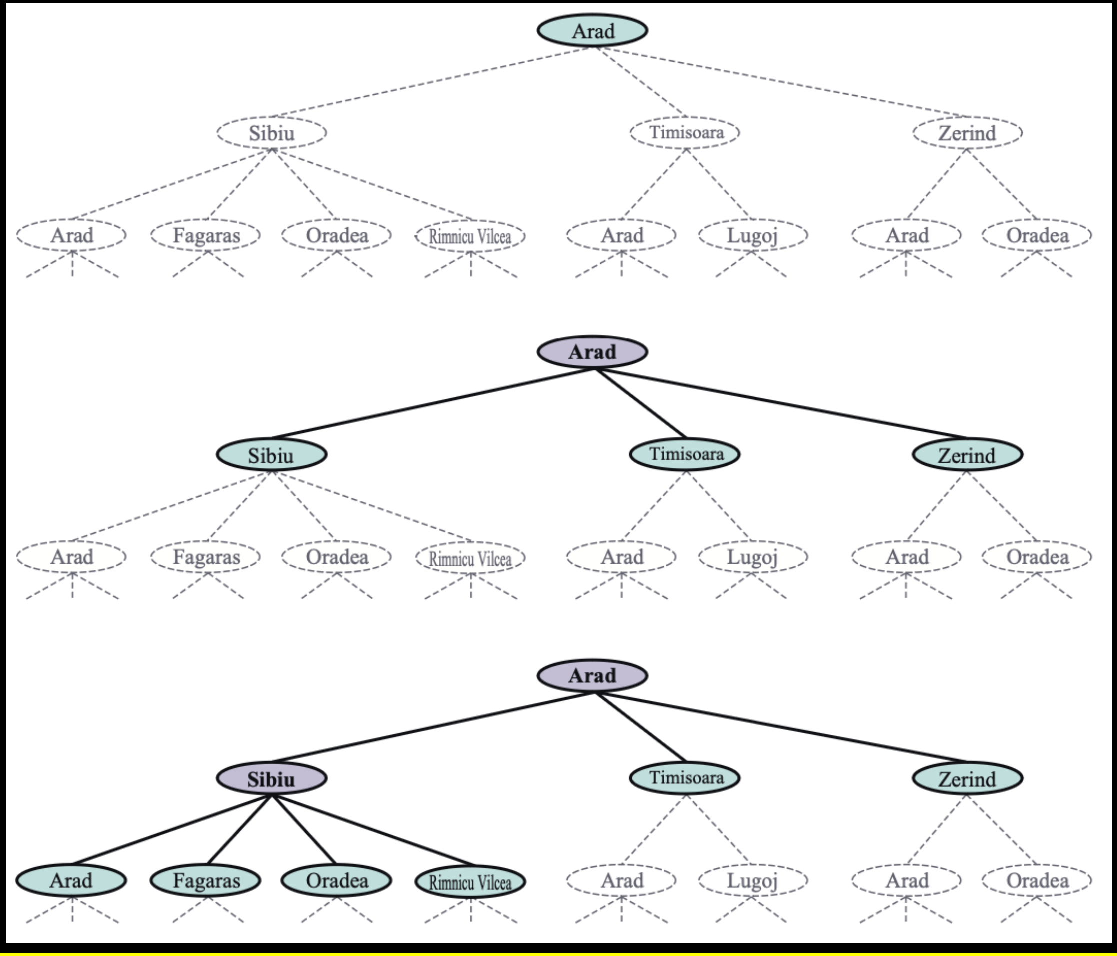
Search Algorithms

A simplified road map of part of Romania, with road distances in miles.



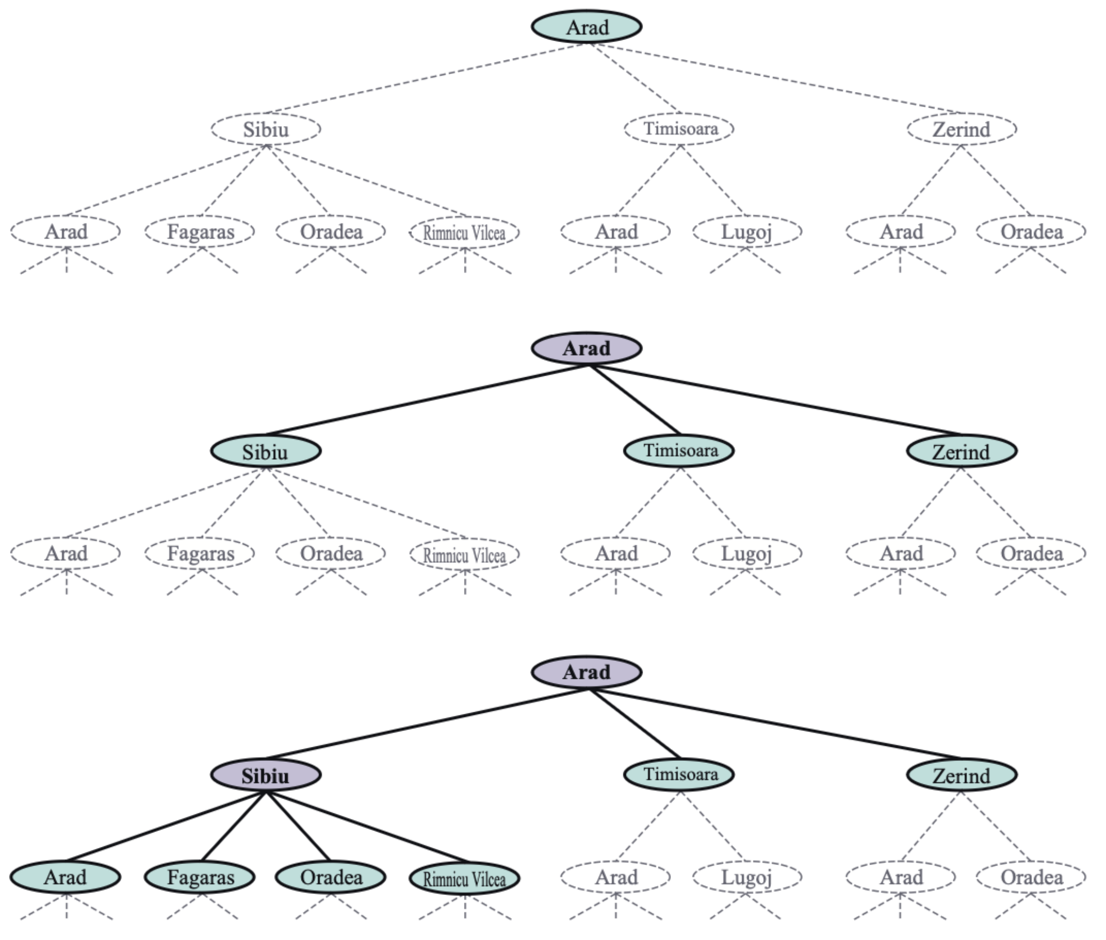
Search Algorithms

- Figure shows the first few steps in finding a path from Arad to Bucharest. The root node of the search tree is at the initial state, *Arad*.
- We can **expand** the node, by considering the available **ACTIONS** for that state, using the **RESULT** function to see where those actions lead to, and generating a new node (called a **child node** or **successor node**) for each of the resulting states.
- Each child node has *Arad* as its parent node.



Search Algorithms

- Now we must choose which of these three child nodes to consider next. This is the essence of search—following up one option now and putting the others aside for later.
- Suppose we choose to expand Sibiu first. Figure shows the result: a set of 6 unexpanded nodes.
- We call this the **frontier** of the search tree. We say that any state that has had a node generated for it has been **reached**.



Search Algorithms

- Three kinds of queues are used in search algorithms:
  - A **priority queue** first pops the node with the minimum cost according to some evaluation function, f. It is used in best-first search.
  - A **FIFO queue** or first-in-first-out queue first pops the node that was added to the queue first; we shall see it is used in breadth-first search.
  - A **LIFO queue** or last-in-first-out queue (also known as a stack) pops first the most recently added node; we shall see it is used in depth-first search.

Search Algorithms

Measuring problem-solving performance

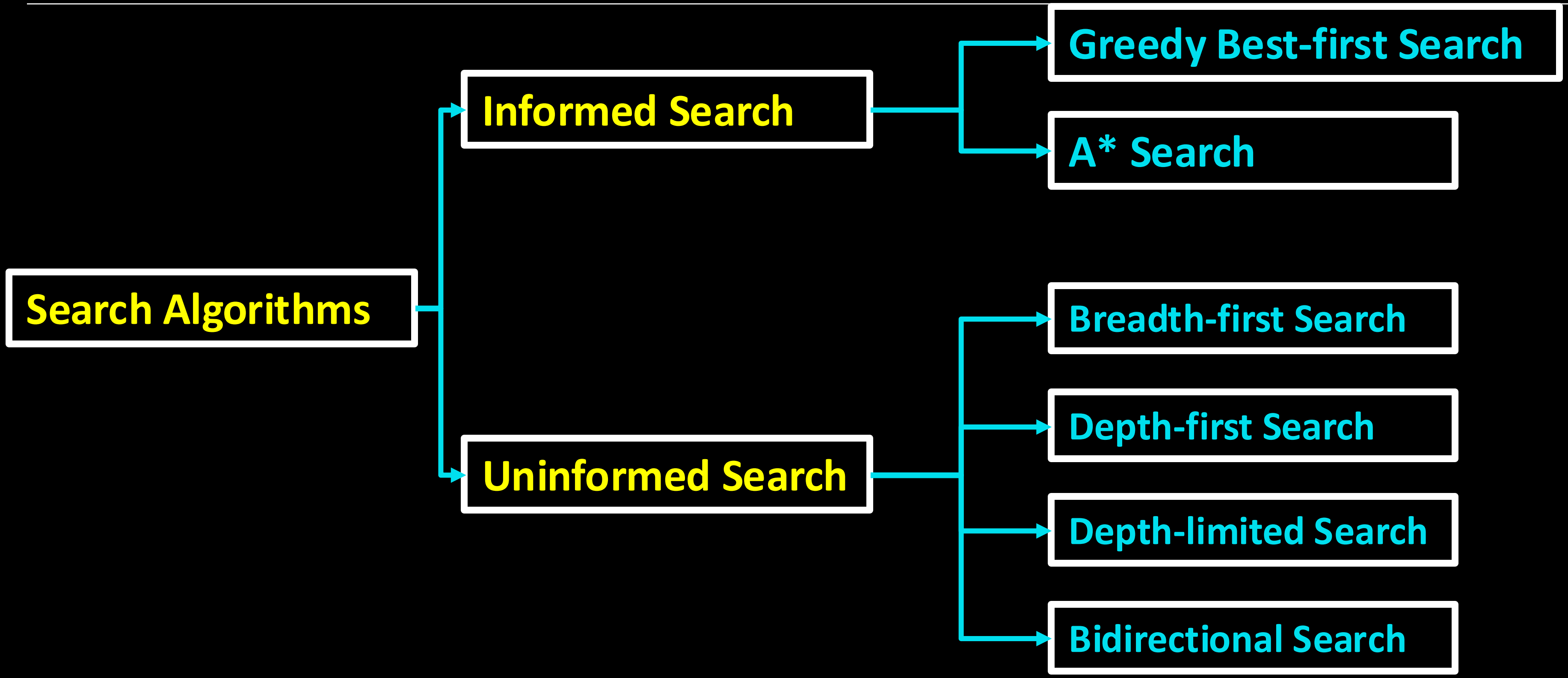
We can evaluate an algorithm's performance in four ways:

- **Completeness:** Is the algorithm guaranteed to find a solution when there is one, and to correctly report failure when there is not?
- **Cost optimality:** Does it find a solution with the lowest path cost of all solutions?
- **Time complexity:** How long does it take to find a solution? This can be measured in seconds, or more abstractly by the number of states and actions considered.
- **Space complexity:** How much memory is needed to perform the search?

Content

- ~~Search Algorithms~~
- Uninformed Search Strategies
- Informed (Heuristic) Search Strategies

Search Algorithms

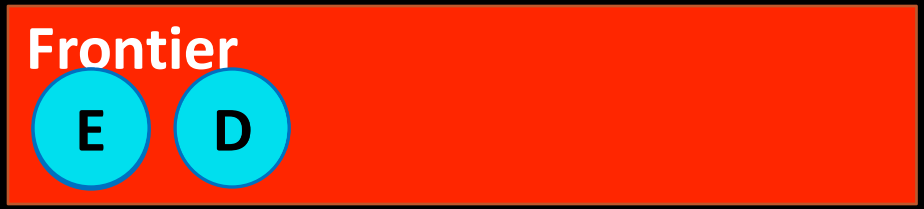


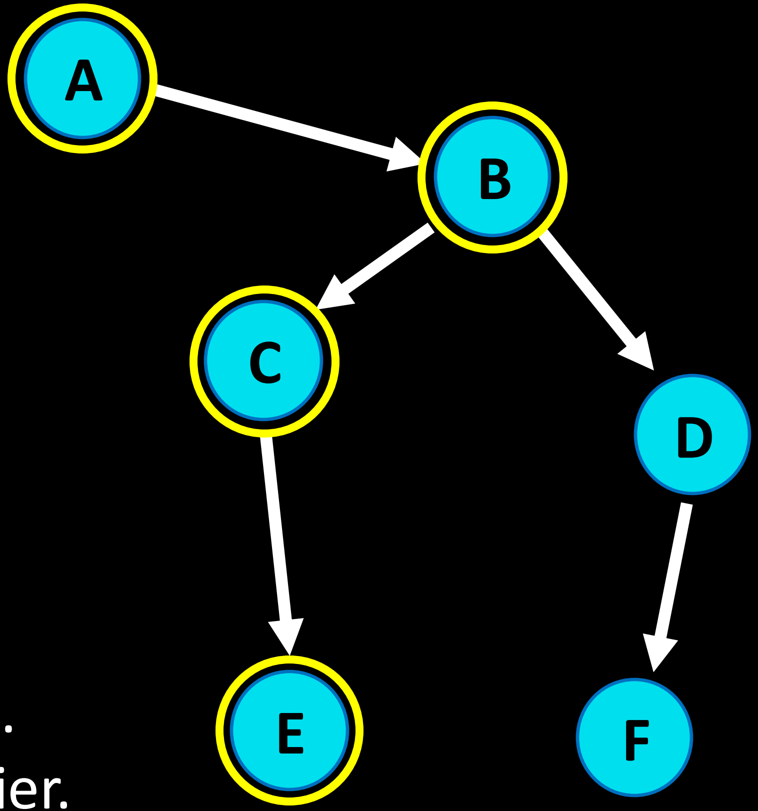
Uninformed Search Strategies

- An **uninformed search** algorithm is given no clue about how close a state is to the goal(s).
- For example, consider our agent in Arad with the goal of reaching Bucharest.
- An uninformed agent with no knowledge of Romanian geography has no clue whether going to Zerind or Sibiu is a better first step.

Search Strategies Representation

Find a path from A to E.





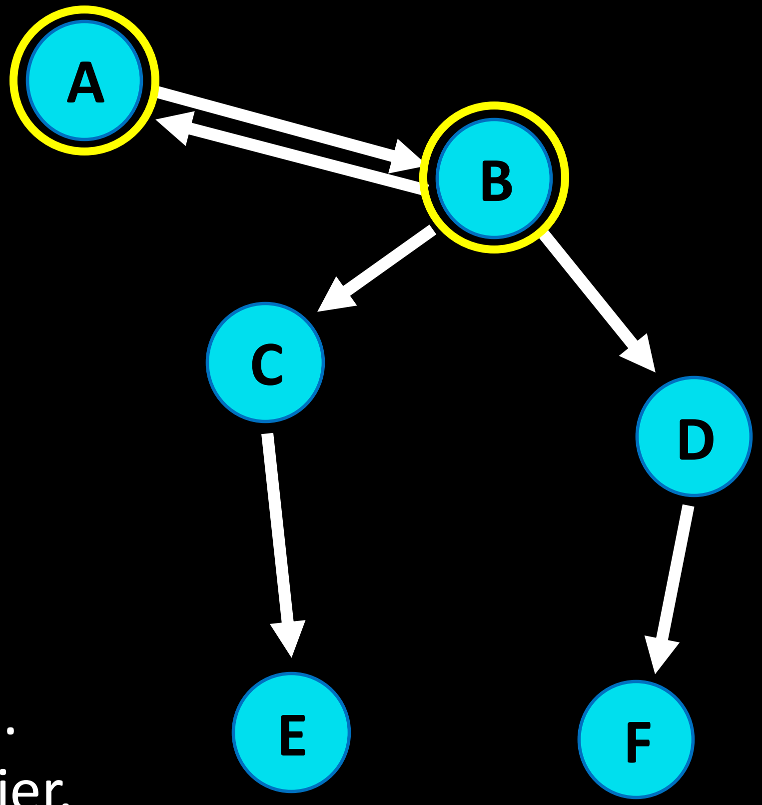
- Start with a frontier that contains the initial state.
- Repeat:
  - If the frontier is empty, then no solution.
  - Remove a node from the frontier.
  - If node contains goal state, return the solution.
  - Expand node, add resulting nodes to the frontier.

Search Strategies Representation

Find a path from A to E.

Frontier

A C D



- Start with a frontier that contains the initial state.
- Repeat:
  - If the frontier is empty, then no solution.
  - Remove a node from the frontier.
  - If node contains goal state, return the solution.
  - Expand node, add resulting nodes to the frontier.

Search Strategies Representation

Revised Approach

- Start with a frontier that contains the initial state.
- Start with an empty explored set
- Repeat:
  - If the frontier is empty, then no solution.
  - Remove a node from the frontier.
  - If node contains goal state, return the solution.
  - Add the node to the explored set.
  - Expand node, add resulting nodes to the frontier if they aren't already in the frontier or the explored set.

Search Strategies Representation

Find a path from A to E.

Stack

Last-in first-out data type

Search Strategies Representation

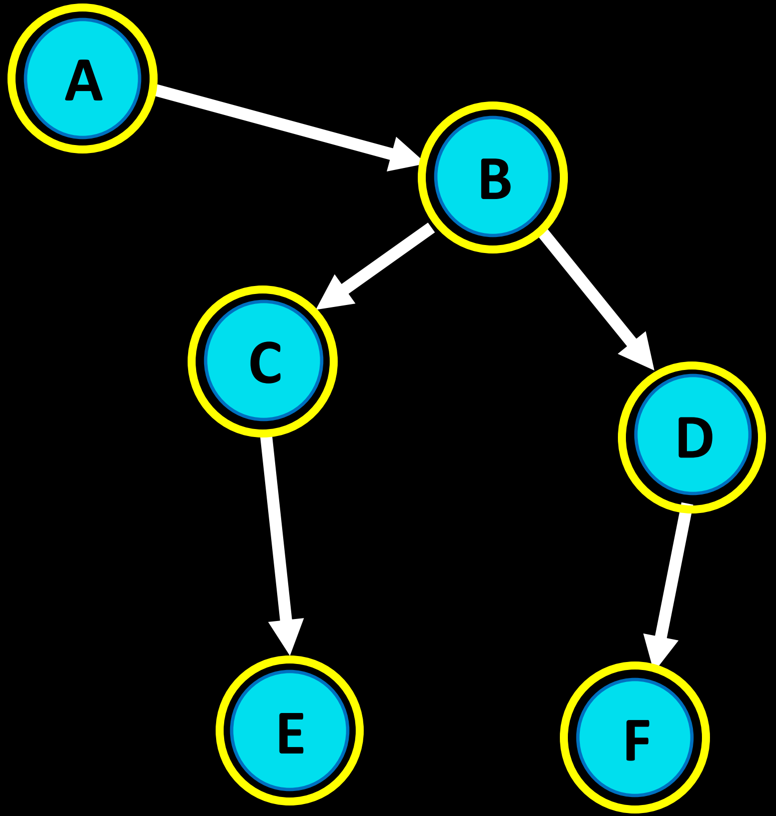
Find a path from A to E.

Frontier

E F

Explored Set

A B D F C



Search Strategies Representation

Find a path from A to E.

Depth-first Search

Search algorithm that always expands the **deepest** node in the frontier

Search Strategies Representation

Find a path from A to E.

Breadth-first Search

Search algorithm that always expands the **shallowest** node in the frontier

Search Strategies Representation

Find a path from A to E.

---

queue

First-in first-out data type

Uninformed Search Strategies

Breadth-first Search

- When all actions have the same cost, an appropriate strategy is **breadth-first search**, in which the root node is expanded first, then all the successors of the root node are expanded next, then *their* successors, and so on.
- This is a systematic search strategy that is therefore complete even on infinite state spaces.

Search Strategies Representation

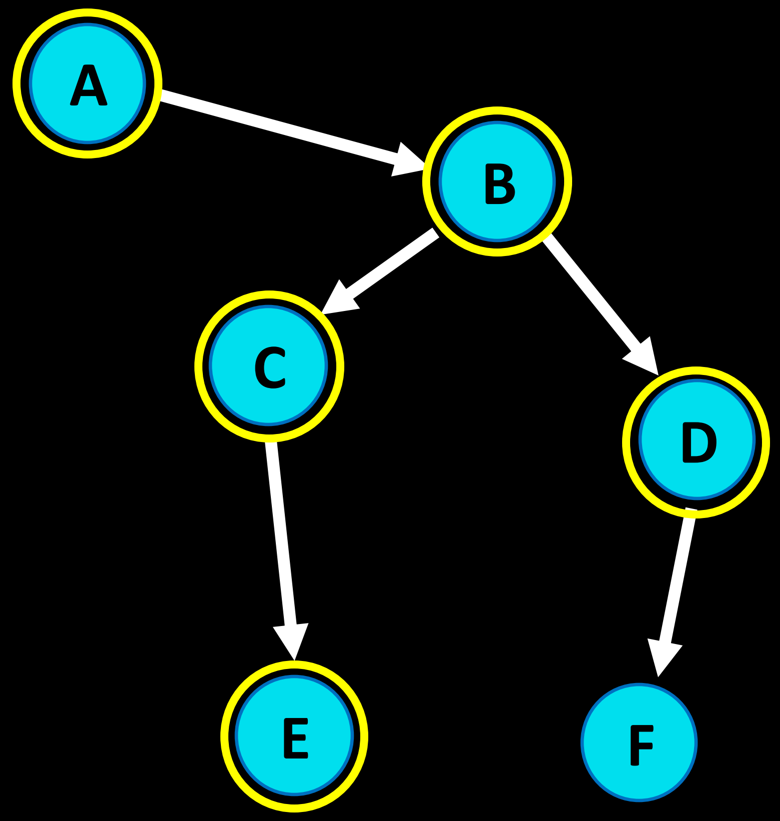
Find a path from A to E

Frontier

C D E F

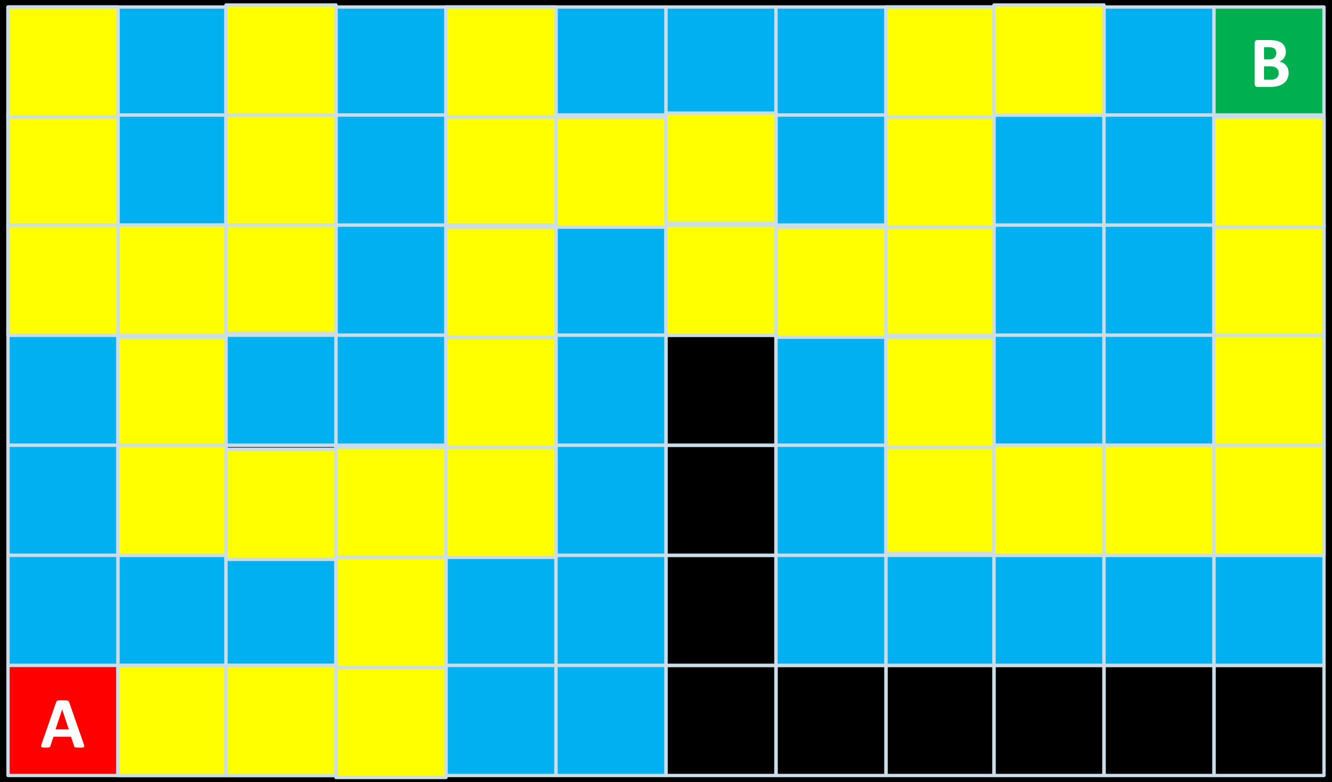
Explored Set

A B C D



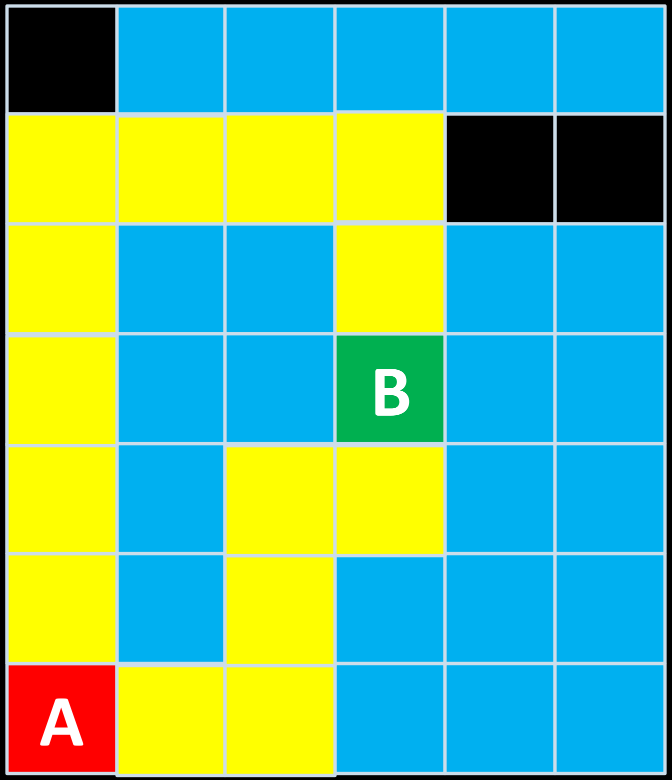
Search Strategies Representation

Maze problem -> Depth-First Search



Search Strategies Representation

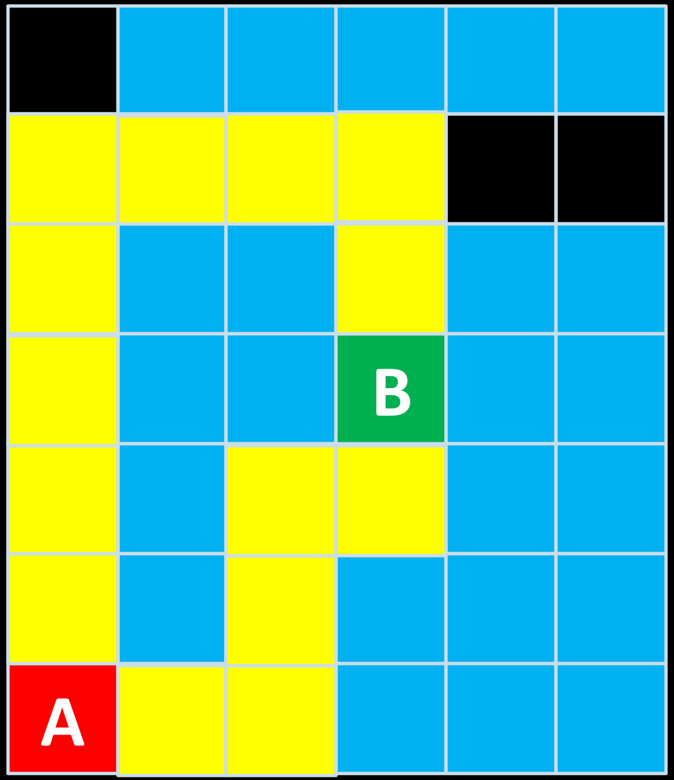
Maze problem -> Depth-First Search



Depth-first search does not guarantee to find the best solution.

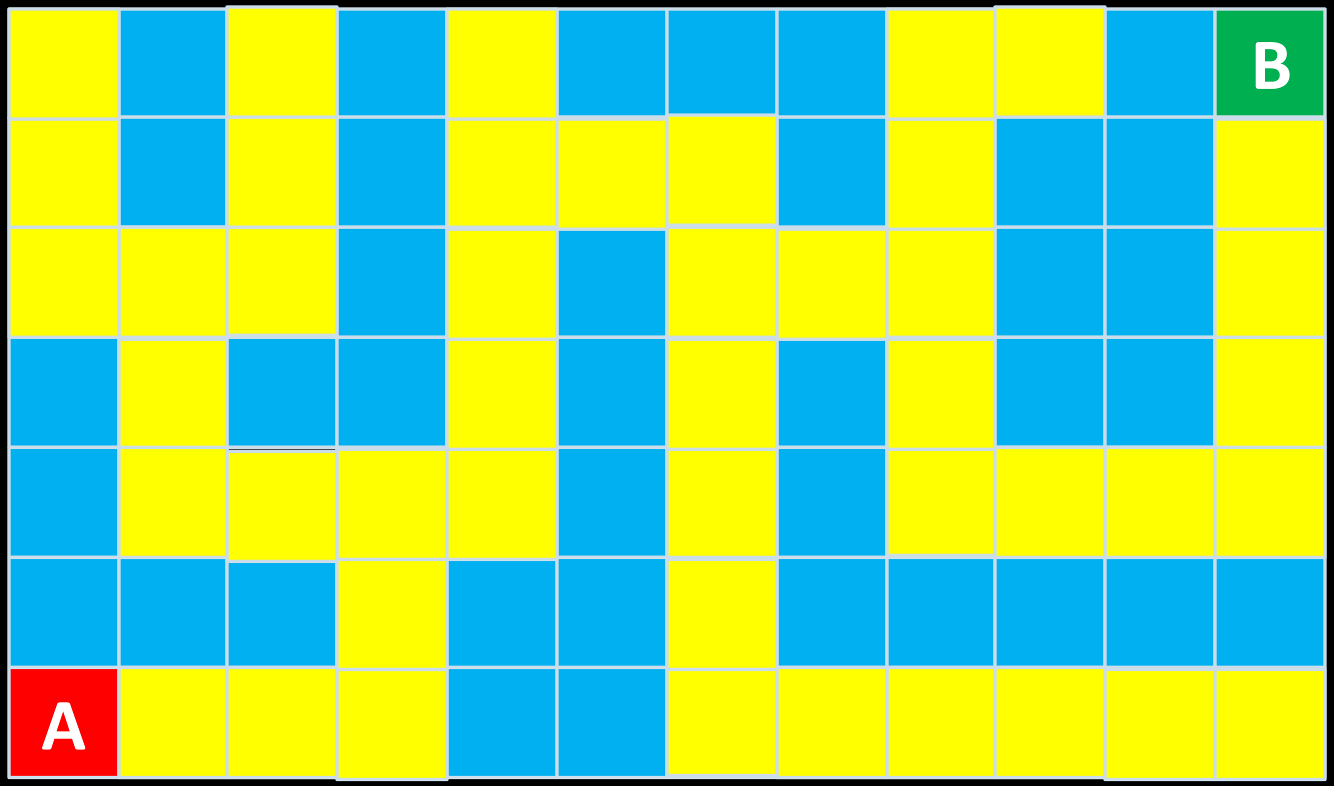
Search Strategies Representation

Maze problem -> Breadth-First Search



Search Strategies Representation

Maze problem -> Breadth-First Search



Content

- ~~Search Algorithms~~
- ~~Uninformed Search Strategies~~
- Informed (Heuristic) Search Strategies

Informed (Heuristic) Search Strategies

- **Uninformed search**

  Search strategy that uses no problem-specific knowledge

- **Informed search**

  Search strategy that uses problem-specific knowledge to find solutions more efficiently

Informed (Heuristic) Search Strategies

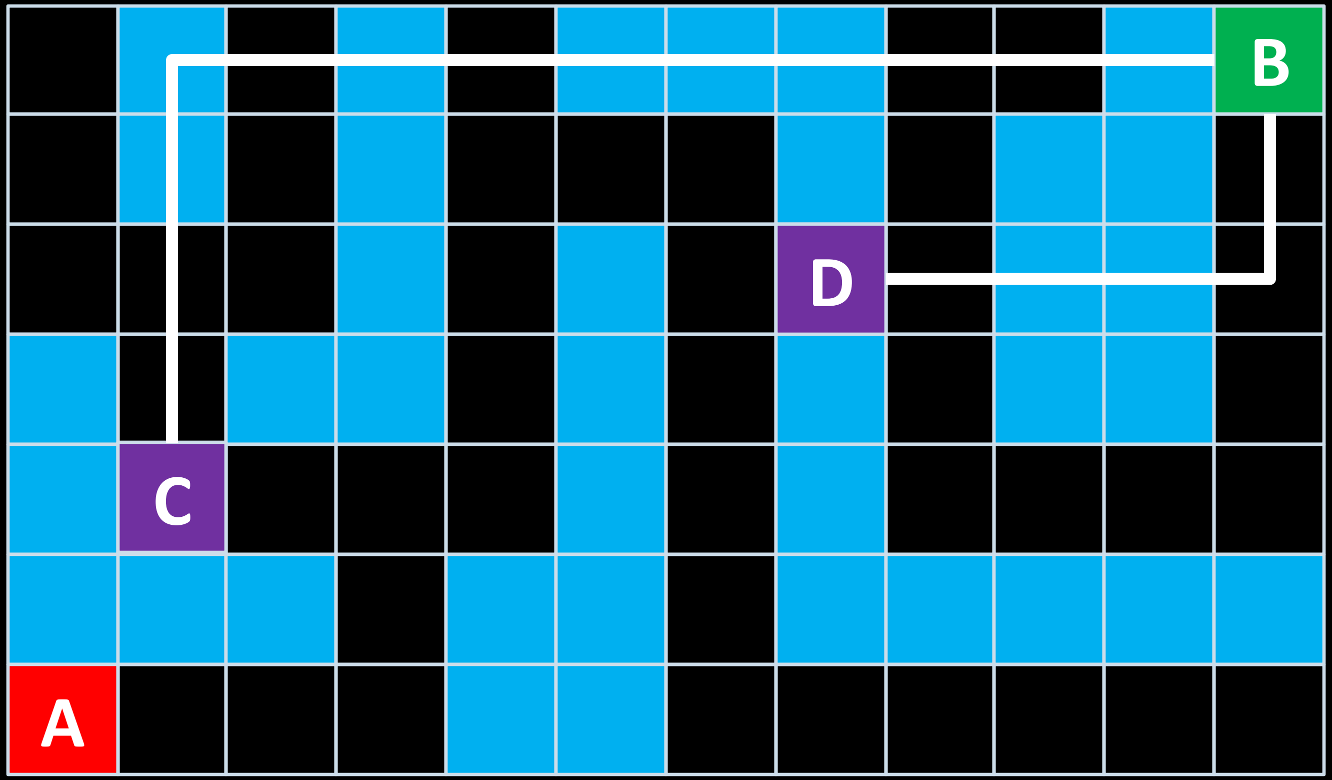
Greedy best-first search

- **Greedy best-first search** is a form of best-first search that expands first the node with the lowest $h(n)$ value—the node that appears to be closest to the goal—on the grounds that this is likely to lead to a solution quickly.

- So the evaluation function $f(n) = h(n)$.

Informed (Heuristic) Search Strategies

Heuristic function. Manhattan distance.



Informed (Heuristic) Search Strategies

Heuristic function. Manhattan distance.

11		9		7				3	2		B
12		10		8	7	6		4			1
13	12	11		9		7	6	5			2
	13			10		8		6			3
	14	13	12	11		9		7	6	5	4
			13			10					
A	16	15	14			11	10	9	8	7	6

Informed (Heuristic) Search Strategies

Greedy best-first search

11		9		7				3	2		B
12		10		8	7	6		4			1
13	12	11		9		7	6	5			2
	13			10		8		6			3
	14	13	12	11		9		7	6	5	4
			13			10					
A	16	15	14			11	10	9	8	7	6

Informed (Heuristic) Search Strategies

Greedy best-first search

	10	9	8	7	6	5	4	3	2	1	B
	11										1
	12		10	9	8	7	6	5	4		2
	13		11						5		3
	14	13	12		10	9	8	7	6		4
			13		11						5
A	16	15	14		12	11	10	9	8	7	6

Informed (Heuristic) Search Strategies

Greedy best-first search

	10	9	8	7	6	5	4	3	2	1	B
	11										1
	12		10	9	8	7	6	5	4		2
	13		11						5		3
	14	13	12		10	9	8	7	6		4
			13		11						5
A	16	15	14		12	11	10	9	8	7	6

Informed (Heuristic) Search Strategies

A* search

A* search algorithm that expands node with lowest value of $g(n)+h(n)$

$g(n)$ = cost to reach node
$h(n)$ = estimated cost to goal

Informed (Heuristic) Search Strategies

A* search

	11+10	12+9	13+8	14+7	15+6	16+5	17+4	18+3	19+2	20+1	B
	10+11										1
	9+12		7+10	8+9	9+8	10+7	11+6	12+5	13+4		2
	8+13		6+11						14+5		3
	7+14	6+13	5+12		10	9	8	7	15+6		4
			4+13		11						5
A	1+16	2+15	3+14		12	11	10	9	8	7	6

Content

- ~~Search Algorithms~~
- ~~Uninformed Search Strategies~~
- ~~Informed (Heuristic) Search Strategies~~


THANK YOU